
DR. JARKKO MOILANEN

AI PRODUCTS AND DATA PRODUCTS

Two Product Contracts, One Connected Portfolio

A fact-based framework for separating, combining and managing AI products and data products.



Table of contents

1. The Problem Is Not Terminology Alone	4	16. The Open Data Products Standards Family as a Machine-Readable Product Layer	25
2. Start With the Word Product	5	17. AI Agents Make the Distinction More Important	27
3. Data Product Has More Than One Legitimate Meaning	6	18. From Separate Inventories to Connected Product Portfolios	29
4. AI Product Needs to Be Derived More Carefully	8	19. Maysano as an Implementation Example of the Connected Model	31
5. The Similarities Are Real	10	20. When the Distinction Matters	32
6. The Decisive Question: What Is Being Productized?	11	21. When the Distinction Adds Little Value	33
7. Two Product Contracts	13	22. Decision Framework	34
8. The Relationship Is Better Described as a Graph Than a Hierarchy	15	23. Worked Examples	35
9. The Removal Test	17	24. Organizational Language That Reduces Confusion	37
10. Four Primary Classifications	19	25. Implications for Product Portfolios	38
11. Failure Modes Reveal the Product Boundary	20	26. Final Position	39
12. The Lifecycles Are Related but Not Identical	21	Appendix A. Detailed Comparison Matrix	40
13. Different Quality Systems	22	Appendix B. Product Qualification Checklist	41
14. Governance: Information Control and Behaviour Control	23	Appendix C. Data Consumption Contract Checklist	42
15. Economics and Value Creation Also Differ	24	Appendix D. Inference Contract Checklist	43
		Appendix E. Source Notes and Evidence Base	44

Abstract

The terms AI product and data product now appear across product management, data platforms, machine learning, generative AI and enterprise architecture, yet they are not used consistently. In one organization, a data product may mean a governed domain dataset or reusable API; in another, the term may encompass dashboards, analytical applications, predictive models and complete decision-support solutions. The term AI product is equally elastic, covering everything from a model endpoint to a copilot, an autonomous agent or an established software product that has acquired one AI-enabled feature. This variation is more than a linguistic inconvenience because the labels influence ownership, architecture, governance, service expectations, metrics, funding and the way portfolios are structured.

This white paper argues that AI products and data products should not be treated as mutually exclusive technology categories. A more useful distinction emerges when the analysis starts from what has been productized for a consumer. A data product primarily productizes data or information for recurring consumption, whereas an AI product primarily productizes inference or AI-generated behaviour. The distinction therefore rests on the value contract with the consumer rather than on the technology stack. A data product promises dependable access to reusable, understandable and governed information. An AI product promises useful behaviour produced through inference, including prediction, recommendation, classification, generation, reasoning or action.

These contracts frequently coexist. An AI product may consume one or more data products, and the outputs of that AI product may themselves become reusable data products for downstream consumers. A broad data product may also contain AI components without becoming, in every case, a separate AI product. For that reason, this paper proposes a practical classification model with four outcomes: data product, AI product, both, and neither yet. It also develops the Removal Test, which examines what independent value remains when inference or the reusable data offering is removed.

The argument is grounded in authoritative AI definitions from the OECD, NIST, ISO and the European Union, peer-reviewed data-product research, data-mesh literature, major technology-provider guidance and established product-management thinking. The paper also examines the Open Data Products standards family as a concrete machine-readable implementation of modern data-product productization. Dr. Jarkko Moilanen, the author of this paper, ignited the Open Data Product Specification work that later expanded into the current Linux Foundation Open Data Products standards family. Maysano is included only as an implementation example of the connected portfolio model, not as independent evidence for the definitions proposed here.

The central conclusion is that organizations gain more from making product contracts and relationships explicit than from trying to force every initiative into a single exclusive category. A connected portfolio should show which business objectives and use cases are served, which data and AI products participate, what each product promises, how they depend on one another, how they are governed and how their contribution to outcomes is measured.

The Problem Is Not Terminology Alone

The industry increasingly uses AI product and data product inside the same organizations, portfolios and architectures. The resulting debate is often framed as a simple classification question: *Is this an AI product or a data product?* That formulation appears practical, but it quietly assumes that the two categories are mutually exclusive. The available evidence does not justify that assumption, and forcing the distinction too early often hides more useful questions about consumer value, accountability and operating commitments.

Formal AI definitions begin from technical behaviour. The OECD defines an AI system around its ability to infer from inputs how to generate outputs such as predictions, content, recommendations or decisions. NIST uses closely related language in the AI Risk Management Framework, while the European Union adopts an inference-centred definition in the AI Act. These definitions establish what makes an AI system an AI system, but they do not by themselves establish when the system should be treated as a product, how broad that product boundary should be, or whether the AI capability is a component inside a larger offering. [S01 S02 S03](#)

Data-product literature begins from a different set of concerns. It concentrates on recurring information needs, consumption, ownership, reuse, quality, packaging, discoverability and value. Peer-reviewed research describes a data product as a managed artifact that satisfies recurring information needs and packages relevant data into consumable form. Data mesh adds domain ownership, product thinking and independently consumable interfaces, while industry definitions range from narrowly governed reusable datasets to broad analytical packages that may contain dashboards, reports, models and pipelines. [S07 S08 S09 S10 S11](#)

The different starting points are significant because they describe different dimensions. AI is formally distinguished by inference and output behaviour, while data products are mainly distinguished by managed information consumption and reuse. These dimensions can exist separately, but they can also coexist in one business initiative. A Customer 360 product, for example, may provide reusable governed customer information to marketing, service, analytics and operational systems. A next-best-action service may consume that information and produce recommendations. The first product promises dependable information; the second promises useful inferred behaviour. They are tightly related without being the same contract.

This paper therefore uses the term product contract to describe the durable promise made to identifiable consumers about value, interface, quality and operating expectations. The term is an author synthesis rather than an established standard, but it provides a practical lens for distinguishing products that share technology, teams or platforms. A data product and an AI product may sit in the same application or even be owned by the same team while still carrying different product contracts and requiring different evidence that those contracts are being met.

Start With the Word Product

Before distinguishing an AI product from a data product, it is necessary to establish what makes an artifact a product at all. Much of the current confusion comes from promoting technical objects to product status simply because they are visible, complex or strategically fashionable. A database table, trained model, dashboard, prompt, prediction endpoint, Kafka topic or pipeline may be important, but none of these artifacts becomes a product solely because it exists or uses advanced technology.

Established product thinking provides a more demanding baseline. Marty Cagan expresses the relationship compactly as “Product = Customer × Business × Technology.” ^{S17} The formulation matters because it rejects a technology-only definition. A successful product has to create value for a user or consumer, make sense for the organization that funds or provides it, and remain technically feasible and operable. Teresa Torres reaches a similar conclusion through continuous product discovery, arguing that product work is organized around outcomes and learning rather than treating shipment as evidence of success. Deployment starts a new phase of product responsibility rather than ending it. ^{S18}

Recent data-product research applies the same logic to enterprise data. Hasan, Finkel and Legner frame data-product design around desirability, feasibility and viability, showing that technically correct data solutions still fail when they are weakly connected to recurring consumer needs or sustained organizational value. ^{S08} Google’s People + AI guidance reaches the same conclusion from the AI side by beginning with user needs, success criteria, data, mental models, trust, feedback and failure rather than with the choice of model. ^{S16} Across these traditions, the common message is that technical sophistication is insufficient evidence of product status.

2.1 Product qualification criteria

For this paper, product status is best understood as a combination of recurring consumption and durable accountability. An artifact begins to behave like a product when identifiable consumers rely on it for a recurring need or outcome, when its value proposition can be explained independently of the implementation, when an owner or team is accountable for its evolution, and when consumers interact with it through defined interfaces or experiences. Product status also implies that quality and service expectations can be articulated, that the offering has a lifecycle beyond one-time delivery, and that usage, outcomes and feedback influence future decisions. Continued investment should have an organizational or economic rationale, and changes should be managed because consumers depend on the offering.

These characteristics apply equally to internal and external products. An internal fraud-scoring service, finance semantic layer or customer data product may serve employees, applications, AI systems or downstream teams rather than paying customers. What matters is that the consumer relationship and operating commitment are durable. This distinction separates artifacts from products without diminishing the importance of the artifacts themselves.

A model checkpoint, for example, is an artifact until it becomes part of a maintained service or experience with consumers, operating commitments and outcome accountability. A source table is an artifact until it is intentionally packaged, documented, governed and supported for repeated consumption. A one-off dashboard is usually a deliverable, while a continuously maintained analytical interface with stable consumers, service expectations and decision outcomes may qualify as a product under a broad data-product interpretation. The product test therefore comes before the AI-versus-data-product test. If the artifact does not qualify as a product, arguing about which product category it belongs to adds little value.

Data Product Has More Than One Legitimate Meaning

There is no single universally accepted definition of a data product, and that ambiguity should be treated as part of the evidence rather than hidden behind a convenient definition. The term has several historical and practical lineages, and those lineages lead to genuinely different product boundaries. Understanding them is essential because the boundary selected for “data product” directly affects how much conceptual overlap exists with AI products.

3.1 The information-product lineage

Product thinking around information predates data mesh by decades. Academic work in the 1990s already treated information as something produced for information consumers, and Richard Wang’s work on Total Data Quality Management helped establish the view that information quality should be assessed in relation to consumer needs rather than only in terms of technical correctness. ^{S19} This lineage matters because it shows that modern data-product thinking did not begin as a platform architecture. It emerged from the persistent organizational problem of producing large volumes of data while still failing to provide information that consumers find understandable, dependable and fit for purpose.

The information-product perspective also makes consumer orientation central. If information is a product, its quality cannot be determined solely by the producer. Accuracy matters, but so do relevance, accessibility, interpretability and the consumer’s ability to use the information within a real decision or workflow. That idea reappears throughout later data-product thinking, even when the implementation shifts from reports and warehouses toward APIs, event streams, semantic models and domain-owned products.

3.2 Data mesh made data-as-a-product operational

Zhamak Dehghani’s data-mesh work gave data-as-a-product major industry visibility by embedding it within a broader operating and architectural model. Data mesh combines domain-oriented decentralized ownership, data as a product, self-service data infrastructure and federated computational governance. ^{S09} The data-as-a-product principle changes the responsibility model: domain data is no longer treated merely as exhaust from operational systems or raw material handed to a central analytics function. It is deliberately designed, documented and operated for consumers.

AWS captures this relatively narrow data-mesh interpretation by describing each data product as “a structured, reusable component” that serves a business purpose and can be used independently by other teams. ^{S10} Reuse and independent consumption create a strong boundary because they require the data offering to stand on its own as something consumers can intentionally select and rely on. Under this interpretation, the data product is not simply a table or pipeline; it is the managed boundary through which consumers obtain domain information.

3.3 Peer-reviewed research reinforces the consumption view

The 2024 *Business & Information Systems Engineering* paper *Data products, data mesh, and data fabric* describes the concept as consumption-oriented and uses the phrase “a managed artifact that satisfies recurring information needs.” ^{S07} The broader definition connects value creation with transforming and packaging relevant data into consumable form.

Its importance lies in the combination of recurring need, management responsibility, transformation, packaging and consumer value. A data product, in this framing, is neither raw data nor a one-time analytical output; it is an offering organized around sustained information consumption.

The same peer-reviewed work also demonstrates why a purely technical AI-versus-data taxonomy is difficult. It recognizes data products ranging from relatively basic datasets to offerings that contain descriptive, predictive or prescriptive analytics. ^{S07} In other words, credible academic usage already extends the data-product concept into territory often associated with machine learning and AI. This is evidence that the overlap is not a fringe interpretation created by vendor marketing. It is built into at least some serious attempts to describe the category.

3.4 Industry definitions range from narrow to broad

Industry definitions reinforce the divergence. AWS stays close to the narrower data-mesh interpretation, where the data product is a reusable and independently consumable data component. ^{S10} IBM adopts a broader boundary, describing a data product as a reusable, self-contained package that combines data, metadata, semantics and templates while explicitly allowing datasets, dashboards, reports, machine-learning models, queries and pipelines to appear inside the package. ^{S11}

These are not cosmetic differences. Under the narrow interpretation, the product boundary is centred on governed information access. Under the broad interpretation, an analytical application or predictive model may legitimately sit inside the data product. Both interpretations are used in credible practice, which means that any paper claiming one universal boundary would overstate the evidence.

3.5 A useful working definition

For the purpose of comparing data products with AI products, this paper adopts an operational definition that keeps the consumer contract visible without denying broader industry usage. A data product is defined here as a continuously managed offering that packages data or information for recurring consumption, with identifiable consumers, accountable ownership, defined semantics and interfaces, measurable quality expectations, governance and lifecycle support. This is a proposed working definition, not a claim that the industry has standardized on one formulation.

The value of this definition is practical. It creates a boundary that organizations can use to reason about ownership, service commitments, interoperability and reuse, while remaining compatible with machine-readable product specifications, catalogs, contracts and graph relationships. It also leaves room for predictive or AI-derived information to appear inside the product, provided that the defining consumer promise remains the dependable consumption of information.

AI Product Needs to Be Derived More Carefully

The source landscape for AI product differs from the data-product landscape. The term is widely used, but no equally authoritative standard provides a standalone technical definition of an AI product. There are, by contrast, mature and credible definitions of an AI system. A defensible AI-product definition therefore needs to combine those technical definitions with ordinary product principles rather than treating every model or AI-enabled feature as a product in its own right.

4.1 OECD: inference is central

The OECD's updated definition states that "An AI system is a machine-based system that ... infers, from the input it receives" how to generate outputs. ^{S01} The possible outputs include predictions, content, recommendations and decisions, and the definition recognizes varying degrees of autonomy and post-deployment adaptiveness. The word *infers* is especially important because it distinguishes AI behaviour from a fully predetermined sequence of software rules. The system derives how to produce an output from its inputs, model and context.

This inference-centred definition gives the comparison with data products a strong technical anchor. It does not say that every inferred output is a product, but it identifies the behaviour that becomes distinctive when inference itself is what the consumer relies on. A model inside a larger software application may therefore constitute an AI system without determining the product boundary of the surrounding application.

4.2 NIST: AI requires its own risk treatment

NIST uses a closely related AI-system definition in the AI Risk Management Framework and adds a second point that is central to product operations. AI creates risks that differ in meaningful ways from conventional information systems because behaviour depends strongly on data, context and changing conditions. Training data and real-world input distributions can change, and those changes can affect functionality and trustworthiness in ways that are difficult to anticipate in advance. ^{S02}

The AI RMF is intended to improve the design, development, use and evaluation of AI products, services and systems. ^{S04} This provides strong evidence that AI-based offerings require lifecycle treatment beyond initial deployment. The operational responsibility is not exhausted when the model reaches production; teams must continue to evaluate behaviour, monitor change and connect technical performance to the context in which the AI is used.

4.3 ISO: AI is an organizational management concern

ISO/IEC 42001:2023 extends the issue from technical operation to organizational management. The standard is intended for organizations that develop, provide or use AI-based products and services and addresses responsible development, risk, traceability, transparency, reliability and continuous improvement. ^{S05} It does not define AI product as a separate technical object. Instead, it treats AI-based products and services as contexts in which AI systems must be governed and managed throughout their lifecycle.

That distinction is useful because product status is an organizational commitment, not a property of the model file. The same AI system may be embedded inside a larger product, exposed as a service, or combined with data, workflow and human interaction in ways that create different product boundaries.

4.4 EU AI Act: an AI system can be a component

The European Union's AI Act is also useful for separating system and product boundaries because it explicitly treats AI systems in relation to broader products and safety components. The legal framework does not assume that every AI system is itself the complete end product. ^{S03} A recommendation model embedded inside an e-commerce application may therefore be an AI system while remaining an internal AI component from a product-management perspective. The same capability, if exposed as a maintained recommendation API with its own consumers, service commitments and outcome accountability, may reasonably be managed as an AI product.

A customer-service agent illustrates the same point at a larger boundary. If the agent interprets requests, generates responses, selects tools and completes service tasks, inferred behaviour is central to what the consumer receives. In that context, the product promise depends materially on AI rather than simply being enhanced by it.

4.5 A working definition of AI product

This paper therefore defines an AI product as a continuously managed product whose core customer or operational value materially depends on outputs inferred or generated by one or more AI systems. As with the data-product definition, this is a proposed operating definition rather than a formal industry standard. Its purpose is to create a boundary that remains meaningful as AI becomes embedded into ordinary software.

Three ideas in the definition deserve emphasis. *Continuously managed* separates a product from an experiment or isolated model artifact. *Core value* prevents every product with a small AI feature from being relabelled as an AI product. *Materially depends on inference* links the product boundary back to the strongest formal descriptions of AI behaviour. These criteria support a practical spectrum in which an AI system is the technical inference mechanism, an AI component is that capability embedded inside another product, an AI-enabled product gains value from AI without depending on it for its primary identity, and an AI product places inference-based behaviour at the centre of the consumer promise. If every software product with an AI feature is called an AI product, the term loses its usefulness for architecture, governance and portfolio decisions.

The Similarities Are Real

AI products and data products share enough characteristics that confusion between them is understandable. Both require identifiable consumers, a value proposition, accountable ownership and a continuing reason for investment. Both rely on data and infrastructure, expose interfaces, require documentation and support, and need security, privacy, observability and change management. Both can be internal or external, commercial or shared enterprise capabilities, and both increasingly appear in product catalogs and portfolios. They also share the basic discipline of discovery, feedback and outcome measurement because neither type of product creates value merely by being deployed.

The similarities extend to machine consumption. Data products are increasingly consumed not only by analysts and applications but also by machine-learning systems and AI agents. AI products are likewise consumed through APIs and automated workflows as well as user interfaces. Both benefit from machine-readable metadata because automated consumers need to understand what an offering is, how it should be accessed, what constraints apply and who is accountable for it. This convergence is one reason technology alone provides such a weak basis for classification.

Implementation clues are therefore poor classifiers. Snowflake, Databricks or Kafka do not make an artifact a data product, because the same technologies can host raw assets, pipelines or internal implementation details. GPT or another model API does not automatically make an application an AI product, because the model may support a minor feature that is not central to the product promise. Producing JSON, exposing an API or storing embeddings likewise tells us little about the product category. The stronger boundary sits in the dominant value contract: what the consumer intentionally relies on and what the provider is accountable for maintaining.

The Decisive Question: What Is Being Productized?

The most useful distinction is not whether data or AI is present, because both are normally present in modern AI systems and many advanced data products. The more informative question is what has been deliberately turned into a managed offering for a consumer. For a data product, the answer is primarily the consumption of data or information. For an AI product, the answer is primarily inferred behaviour. This paper summarizes that distinction as data product = productized data consumption and AI product = productized inference. These formulations are analytical shorthand rather than formal standards, but they keep attention on what the consumer depends on rather than on the implementation stack.

6.1 The data-product promise

A consumer of a data product expects the provider to make information available in a form that remains understandable and usable over time. That promise includes the meaning of the product, the entities or measures it represents, the interfaces through which it is exposed, and the schema or structure consumers should expect. It also extends to quality, freshness, ownership, access, lineage, policy, support and the way breaking changes are managed. The consumer may be a person, an analytical workload, an application, a machine-learning system or an AI agent, but in each case the relationship is primarily about dependable information consumption.

This is why a data product is more than a dataset. The product boundary includes enough context and operating responsibility for another consumer to use the information without having to rediscover its meaning, quality or ownership every time. The stronger the dependency and reuse, the more important the product contract becomes.

6.2 The AI-product promise

A consumer of an AI product expects the system to produce useful behaviour in response to an input, context or objective. That promise has to define the task the AI is intended to perform, the kinds of inputs and contexts it supports, the kinds of outputs or actions it may produce, and the criteria by which acceptable behaviour is judged. It also has to address uncertainty, safety boundaries, human oversight, fallback behaviour, latency, cost and the way model, prompt or retrieval changes are evaluated after deployment.

The contract is therefore behavioural rather than merely informational. A scoring service is valuable because of the quality of its scores in a real decision process. A copilot is valuable because of the quality of its assistance. An agent is valuable because of the tasks it completes and the actions it takes. Availability is necessary, but an available system that behaves poorly does not satisfy the product promise.

6.3 A compact comparison

Dimension	Data product	AI product
Core promise	Dependable information consumption	Useful inference-based behaviour
Defining question	Can consumers repeatedly obtain and trust the information?	Can consumers repeatedly rely on the inferred behaviour?

Dimension	Data product	AI product
Typical outputs	Tables, files, events, APIs, graphs, semantic models, metrics	Predictions, recommendations, classifications, generated content, decisions, actions
Typical consumers	Analysts, applications, domain teams, models, agents	End users, employees, applications, workflows, agents
Dominant uncertainty	Data correctness, semantics, freshness, completeness, compatibility	Behavioural correctness, drift, hallucination, calibration, safety, tool/action errors
Lifecycle focus	Sources, transformations, contracts, schemas, quality, publication, compatibility	Models, prompts, retrieval, evaluation, deployment, monitoring, safety, behaviour changes
Primary quality evidence	Data quality and service-level evidence	Behavioural evaluations and outcome evidence
Typical failure question	Was the information wrong, late, inaccessible or misunderstood?	Did the AI infer, generate, recommend or act incorrectly?

The comparison describes dominant concerns rather than exclusive responsibilities. AI products still depend on data quality, while data products still depend on reliable software and infrastructure. The value of the distinction is that it clarifies which promise is primary and therefore which evidence, lifecycle controls and failure modes deserve special attention.

Two Product Contracts

The evidence above leads to the main conceptual contribution of this paper: organizations should distinguish between a Data Consumption Contract and an Inference Contract when those commitments are independently meaningful. The terms are proposed here as an operating model, not as replacements for formal data contracts or AI standards. Their purpose is to make explicit the different promises hidden inside initiatives that are often discussed under a broad “AI and data” label.



Figure 1. Two Product Contracts: productized data consumption and productized inference can coexist.

7.1 The Data Consumption Contract

The Data Consumption Contract is the promise that a consumer can repeatedly obtain usable, understandable and governed information. A mature contract explains what information the product represents, which consumers and recurring needs it serves, which interfaces and schemas apply, and how semantics are defined. It also specifies the expected level of completeness and correctness, freshness and availability, lineage and provenance, access restrictions, usage rights, ownership, support and the way versions or breaking changes are handled. Finally, it connects these operating properties to measures of actual use and value.

The term is intentionally broader than a technical *data contract*. A technical data contract commonly governs producer-consumer interface expectations such as schema, validation rules and compatibility. The Data Consumption Contract proposed here includes that technical layer but extends it to the complete product relationship. It asks whether consumers know what they are consuming, whether they have a reliable route to it, whether they understand the commitments attached to it, and whether an accountable product owner keeps those commitments meaningful as the product evolves.

7.2 The Inference Contract

The Inference Contract is the promise that an AI product will produce behaviour suitable for a defined task and context. A mature contract clarifies the intended task and user, the supported inputs and contexts, the possible outputs or actions, and the evaluation criteria used to judge whether behaviour is acceptable. It also makes uncertainty and known failure modes visible, defines safety constraints and human-confirmation points, controls which tools an agent may use, and sets expectations for grounding or evidence when the use case requires it.

Because AI behaviour changes when models, prompts, retrieval configurations or providers change, the Inference Contract also needs lifecycle controls. It should state how those changes are evaluated, which latency and reliability targets apply, what cost envelope is acceptable, how behaviour is monitored after deployment and what happens when the system cannot satisfy a request safely or with sufficient confidence. As AI products move from passive recommendations toward semi-autonomous or autonomous action, these commitments become more important because behaviour has direct consequences in operational systems.

7.3 Why both contracts matter

The distinction matters because strong performance in one contract does not prove strong performance in the other. A perfectly fresh and well-governed data pipeline does not prove that an AI assistant produces useful answers. A high-scoring model does not prove that the information it consumes is reusable, legally appropriate, semantically stable or dependable for other consumers. When both contracts exist, mature product management needs evidence for both and a shared outcome that connects them. Treating the initiative as one vague “AI/data product” risks hiding which promise failed when the overall outcome deteriorates.

The Relationship Is Better Described as a Graph Than a Hierarchy

A common debate asks whether AI products are a subtype of data products or whether data products are components of AI products. Both interpretations can be valid in specific contexts, but neither works universally because the concepts describe different dimensions. The more accurate representation is a graph in which products consume, produce and depend on one another while retaining their own contracts.

8.1 Pattern 1: data product without AI product

A canonical supplier data service offers a clear example of a data product that requires no AI-product classification. The service may expose trusted supplier entities, attributes, identifiers and relationships through an API or analytical interface. Its consumers care about consistency, meaning, quality and availability. No prediction or inference is necessary for the central value proposition, so the product is best understood as a data product.

8.2 Pattern 2: AI product consuming data products

A fraud-scoring service illustrates the opposite direction of dependency. The service may consume transaction, customer, merchant and historical-label products, each of which provides reusable governed information. The fraud service then generates a probability, score or recommendation that supports an operational decision. The architecture is therefore naturally described as data products feeding an AI product, with the quality of the inference depending partly on the quality of the upstream information products.

8.3 Pattern 3: AI product producing a data product

Demand forecasting shows why the relationship can continue in the other direction. A forecasting service may generate daily predictions for every product-location combination and therefore qualify as an AI product because inference is central to its value. If those forecasts are then published through a governed schema with freshness, availability and support commitments for planning, procurement and logistics systems, the forecast output becomes independently consumable information. In that case the architecture is best understood as data products , AI product , data product.

8.4 Pattern 4: one initiative exposing both contracts

A customer intelligence platform may expose both contracts directly. Its Customer 360 interface may provide governed customer information that remains useful to service, marketing and analytics teams even when the recommendation engine is unavailable. The same initiative may also provide next-best-action recommendations whose consumers rely on inference. The most informative classification is therefore both, because each contract has independent consumers and operating value.

8.5 Pattern 5: AI product using data assets that are not products

The presence of data inside an AI product does not create a data product automatically. A coding copilot may store repository context, prompt history and telemetry, yet those artifacts may never have independent consumers, product ownership or service commitments. They remain implementation data or internal assets unless the organization deliberately productizes them. In such a case the surrounding offering remains an AI product with data dependencies rather than a combined AI-and-data product.

8.6 Pattern 6: data product containing AI as a component

The opposite configuration is also legitimate. A broad analytical data product may contain predictive or prescriptive models, a pattern supported by IBM's definition and by peer-reviewed data-product literature. ^{S07 S11} The existence of a model inside the package does not require a separate AI-product identity. The stronger test is whether the inference capability has a distinct consumer promise, evaluation regime, lifecycle and operating accountability. If it does, dual classification adds value; if it does not, AI can remain a component inside the broader data product.

The Removal Test

Organizations need a practical way to classify initiatives without relying on technology brands, team names or architecture diagrams. This paper proposes the Removal Test, which asks what independent value remains when either inference or the reusable information offering is taken away. The test is intentionally simple because the point is not to create another complex taxonomy. It is to expose which consumer contracts actually exist.

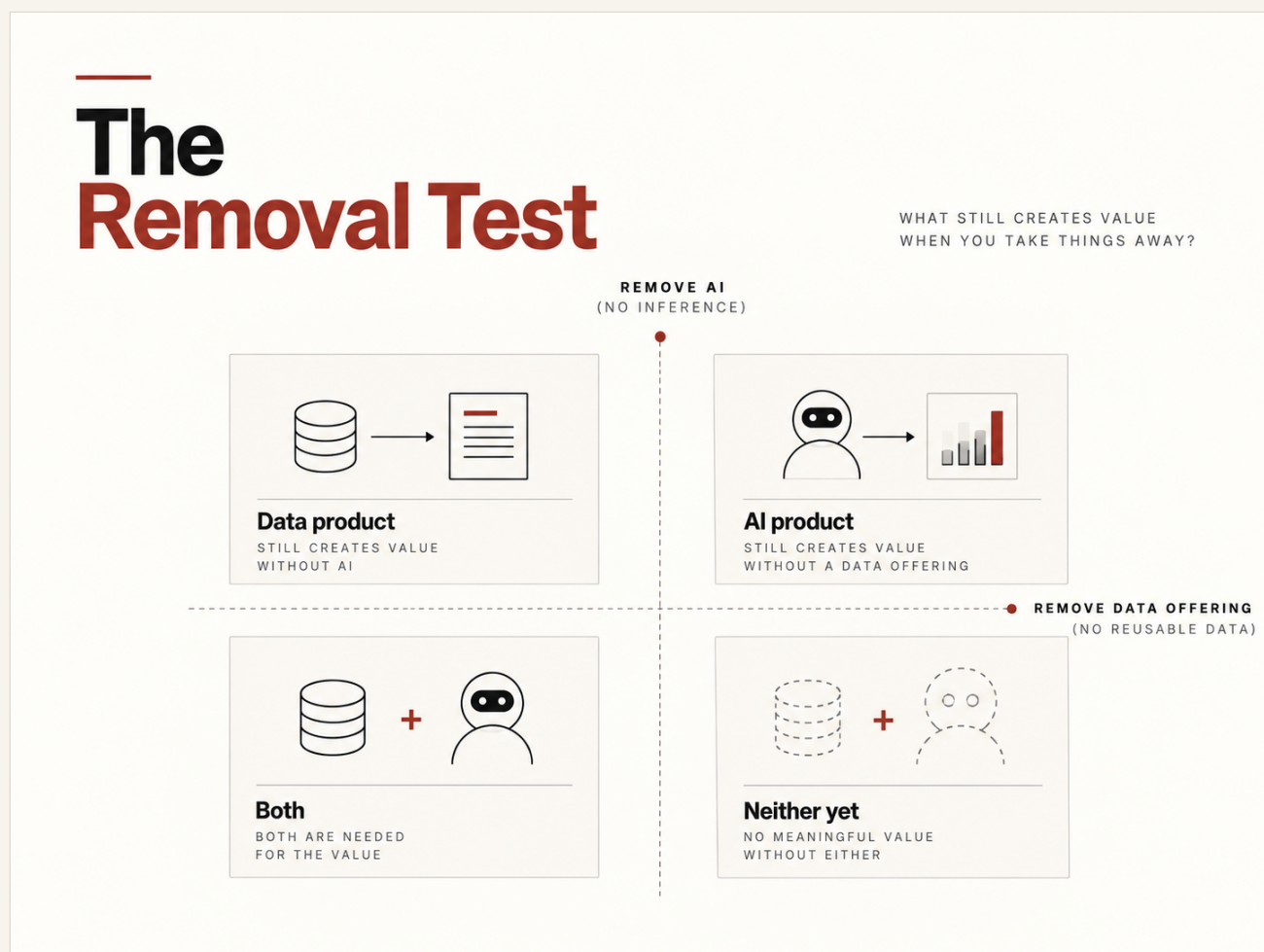


Figure 2. The Removal Test classifies an initiative by asking which value survives when AI inference or a reusable data offering is removed.

9.1 Test A: remove AI inference

The first test removes models, generative capabilities, recommendation engines and other inference services, then asks whether identifiable consumers still intentionally use the remaining offering as a managed source of information. If the answer is yes, there is evidence for a data product because the information contract survives independently of AI. Customer 360 is a straightforward example: marketing, service and analytical consumers may continue to use governed customer information even if a propensity model or recommendation engine is removed.

9.2 Test B: remove inferred behaviour

The second test replaces the AI output with no result or with a static non-AI response and asks whether the central consumer value proposition collapses or materially changes. If it does, there is evidence for an AI product. A coding copilot illustrates the point because the primary value disappears when code generation, transformation and explanation are removed. Internal telemetry or repository context does not create a separate data product merely because it exists inside the architecture.

9.3 Test C: apply both independently

The most useful result often appears when both tests are applied independently. If a governed information offering remains valuable without AI and a separate inference-based capability also carries its own consumer promise, the initiative contains both contracts. A Customer 360 platform with an independently managed next-best-action service fits this pattern, as does a forecasting hub that provides reusable planning data and a separately evaluated forecast-generation service.

9.4 Test D: verify product status

The Removal Test should not promote every surviving artifact to product status. The organization still needs to verify that identifiable consumers, durable ownership, measurable value, supported interfaces, service expectations and lifecycle responsibility exist. If those characteristics are missing, the object is more accurately described as an asset, model, report, experiment, feature or project. A one-off churn notebook therefore remains neither yet, even if its model and training dataset can be discussed separately.

9.5 Reading the results

A published forecast table illustrates a more subtle result. Removing the forecasting model would prevent new forecasts from being generated, which confirms the existence of an AI-product contract around forecast generation. Yet once the forecasts are published through a durable governed interface and consumed by downstream systems, that output may also form a data-product boundary. The Removal Test therefore does not force one label to win. It helps reveal when value exists at more than one product boundary.

Four Primary Classifications

The two-contract model produces four practical classifications, but they should be read as operating descriptions rather than maturity rankings. A data product exists when a durable Data Consumption Contract is present and the reusable information offering has value independent of AI. Canonical customer data, supplier master data, finance semantic layers, governed event streams, market reference feeds and reusable domain APIs fit this pattern when they have consumers, ownership and service commitments.

An AI product exists when a durable Inference Contract is central to the offering and there is no independently valuable data offering that needs separate product treatment. Coding copilots, image-generation services, document classifiers, vision-based defect detectors, recommendation APIs and task-oriented agents often fit this pattern because consumers primarily seek inferred behaviour rather than access to reusable information assets.

The classification both applies when the two contracts are independently meaningful. Fraud intelligence platforms can provide reusable governed fraud data while also operating a separate scoring capability. Demand-forecasting hubs can expose governed planning data and inference services. Enterprise knowledge platforms can maintain a permission-aware knowledge product while also providing an AI assistant. Customer intelligence platforms can combine Customer 360 with next-best-action services. In each case, the reason for dual classification is not that “data and AI are both used,” but that both product promises have their own consumers, quality evidence or operating commitments.

The final classification, neither yet, applies when an artifact has not established a durable product relationship. A training notebook, model checkpoint, raw table, temporary dashboard, prompt prototype, ad hoc extract or experimental agent may be useful and may later become part of a product, but calling it a product prematurely creates a false sense of ownership, service maturity and portfolio value. “Neither yet” therefore protects the meaning of product without judging the technical work itself.

Failure Modes Reveal the Product Boundary

One of the clearest ways to understand the distinction is to examine how each product contract fails. Data-product failures are usually failures of the information supply relationship. Records may be incorrect or missing, data may arrive late or become stale, schemas may break consumers, semantics may be unclear, lineage may be missing, access may fail or policies may be applied incorrectly. Duplication and contradictory definitions can create equally serious problems because consumers no longer know which representation to trust. In each case, the central failure is that the consumer cannot depend on the information as promised.

AI-product failures are dominated by behaviour rather than by information delivery alone. A prediction may be wrong, a recommendation may be poor, generated content may be unsupported or unsafe, calibration may be weak, an agent may select the wrong tool, or model-provider changes may alter behaviour in unexpected ways. Performance can degrade after data drift, latency can make an otherwise accurate system unusable, and inference cost can make a technically successful service economically unsustainable. NIST emphasizes that AI risks differ from traditional software risks in meaningful ways, including their sensitivity to changing data and socio-technical context. ^{S02} Google’s MLOps guidance similarly observes that “the real challenge isn’t building an ML model” but operating an integrated ML system continuously. ^{S14} Microsoft treats model degradation, data drift, prediction drift, security and cost as production monitoring concerns. ^{S15}

The distinction becomes especially useful when both contracts exist because failures then form a chain. An AI system may behave correctly given incorrect source information, while a perfectly fresh and schema-conformant data product may still feed a model that produces a poor recommendation. A model may reason correctly while an agent selects the wrong tool, and the right tool may still fail at authorization or execution. The combined diagnostic chain therefore runs from data quality to context quality, inference quality, action quality and finally outcome quality. Collapsing these layers into a single product quality score hides where intervention is needed and makes accountability harder to assign.

The Lifecycles Are Related but Not Identical

AI products and data products both require continuous operation, but they do not evolve through the same dominant lifecycle. A data product usually begins with a recurring information need and identifiable consumers, after which the team defines product scope, semantics, sources, transformations, interfaces, quality and service expectations. Governance and access are established before publication, and continued operation requires monitoring consumption, freshness, quality and compatibility while evolving schemas and semantics carefully. Retirement becomes a product decision when the need disappears or a successor product replaces the offering. IBM's published lifecycle follows the same general pattern of definition, development, packaging, governance, publication, consumption, monitoring, iteration and retirement. ^{S11}

An AI-product lifecycle begins with the desired user or operational outcome and the question of whether AI is appropriate for achieving it. The team then defines the task and expected behaviour, identifies or prepares required data, selects or configures models, and builds prompts, retrieval or feature logic where relevant. Evaluation sets are created before production use so that quality, safety, latency and cost can be tested against the intended context. Deployment is followed by monitoring of both outcomes and behaviour, detection of drift or degradation, controlled evaluation of model, prompt and retrieval changes, and eventual retraining, replacement or retirement of components. Audit and governance evidence must persist across that lifecycle rather than being assembled only for a release gate.

Google's MLOps guidance reinforces this operational difference by describing automation and monitoring across integration, testing, release, deployment and infrastructure management while emphasizing that model code is only a small part of a real-world ML system. Data collection, verification, serving, metadata, monitoring and resource management surround the model. ^{S14} This matters for portfolio governance because an organization that applies the same generic lifecycle template to every product risks missing essential controls. A data-product review that focuses on schema and freshness will not establish whether a generative assistant behaves safely, while an AI-product review that focuses on model evaluations will not establish whether a shared customer interface has stable semantics or compatibility commitments. Shared governance is valuable, but identical governance is not always sufficient.

Different Quality Systems

Quality provides another clear example of overlap without equivalence. Data-product quality is concerned with the reliability of the information supply relationship. Accuracy, completeness, validity, consistency, timeliness, freshness, uniqueness and availability all matter, but so do interpretability, lineage completeness, schema stability and semantic consistency. These properties tell consumers whether the information is dependable enough to use and whether its meaning will remain stable as the product evolves.

AI-product quality is more task-dependent because the product promise is behavioural. Classification services may rely on precision, recall and calibration; ranking systems may need ranking quality; forecasts require forecast-error measures; generative systems may need grounded-answer rates, human preference or policy-violation measures; and agents may need task-completion and tool-call success metrics. Latency, fallback rates, harmful-output rates and cost per successful task may be as important as an offline model score. NIST's trustworthiness characteristics include validity, reliability, safety, security, resilience, accountability, transparency, explainability, privacy and fairness considerations, which reinforces the point that no single model metric proves product quality. **S02 S04**

When both contracts exist, the two quality systems should be connected through a shared outcome rather than merged into one scorecard. A customer-service initiative, for example, may aim to reduce handling time. The underlying customer data product might be measured through freshness, profile completeness and API availability, while the AI assistant is measured through correct-answer rate, tool-execution success, escalation rate and latency. The business outcome connects the layers, while the separate operational measures identify which product contract is responsible when performance changes.

Governance: Information Control and Behaviour Control

Governance offers another reason to preserve the distinction because the objects being governed are related but not identical. Data-product governance is primarily concerned with the controlled and trustworthy use of information. Ownership, classification, access, permitted use, retention, lineage, quality, schema management, interoperability, licensing, privacy, documentation and lifecycle state all belong close to the product contract because consumers need to know what they are allowed to use and what they can rely on.

AI-product governance adds behavioural and contextual concerns. Intended purpose and risk classification matter because the same model can create different consequences in different settings. Evaluation evidence, model and data provenance, human oversight, acceptable failure rates, fairness and explainability where relevant, safety, security, model-provider changes, incident handling, auditability, autonomous actions and tool permissions all become part of the operating model. ISO/IEC 42001 provides an organizational management framework for AI risk, responsibility and continuous improvement, while NIST treats AI risk management as a lifecycle activity rather than a one-time technical approval. **S05 S04**

The two governance systems therefore need to connect without becoming indistinguishable. A data product may be perfectly governed as information and still be used by an AI system in an unacceptable context. An AI system may satisfy behavioural controls while consuming data without the necessary rights or retention permissions. Mature governance consequently asks two linked questions: whether the organization is allowed and able to supply the information in the proposed way, and whether it is willing and able to rely on the inferred behaviour in the intended context. Treating these as separate but connected obligations produces clearer accountability than combining them into one generic checklist.

Economics and Value Creation Also Differ

The distinction between product contracts has financial consequences because data products and AI products tend to create value through different mechanisms. Data products often create value through reuse, reduced duplication, faster consumer onboarding, lower integration effort, consistent semantics and less reconciliation work. Better trust and governed access can reduce the time required to produce analytics or launch downstream products, while some data products also support external monetization. Their cost base is often dominated by storage, movement, transformation, platform usage, quality management, governance, cataloging, support and domain ownership.

AI products often create value through automation, augmentation, prediction, decision improvement, personalization, reduced manual effort, faster task completion and autonomous execution. Their economics increasingly include inference, model APIs, accelerators, retrieval, vector storage, evaluation, human review, safety controls, orchestration, monitoring and tool execution, in addition to conventional software and data costs. These costs can scale differently from storage-centric data-platform costs because each user interaction or agent action may incur incremental inference and orchestration expense.

The different value mechanisms suggest different unit economics. A data product may justify investment by showing reuse across many teams, reduced integration time or avoided duplication. An AI product may be better measured through cost per successful task, incremental revenue, avoided loss or the value of decisions improved. When an initiative contains both contracts, their economics should remain visible rather than being hidden inside one undifferentiated platform budget. That transparency helps portfolio leaders decide whether investment is strengthening shared information infrastructure, inference capability, or both.

The Open Data Products Standards Family as a Machine-Readable Product Layer

The conceptual distinction in this paper becomes more actionable when organizations can represent product contracts in machine-readable form. The Open Data Products standards family, developed under the Linux Foundation, provides one implementation of that principle. Dr. Jarkko Moilanen, the author of this paper, ignited the Open Data Product Specification work, which later expanded into the current Open Data Products standards family with maintainers and contributors across the community. The public project site identifies Moilanen as igniter and maintainer across ODPS, ODPC, ODPV, ODPR and the AI Agent SDK, while ODPG is maintained within the wider family by additional maintainers. ^{S20}

16.1 ODPS: the product

The Open Data Product Specification (ODPS) defines the individual data product in a vendor-neutral, open-source and machine-readable form. The current 4.1 specification covers product information together with operational concerns such as access, quality, service levels, licensing, pricing and strategy. ^{S20 S21} In the language of this white paper, ODPS provides a concrete way to express many parts of the Data Consumption Contract instead of leaving them in human-readable documentation scattered across tools.

16.2 ODPC: the catalog and portfolio

The Open Data Product Catalogs (ODPC) specification extends the focus from one product to collections and portfolios. It organizes products into catalogs and connects them with use cases, objectives, KPIs and signals. ^{S20} That shift is important because the value of a data product is rarely visible from its metadata alone. Portfolio context shows why the product exists, which outcomes it supports and how it participates in a wider product system.

16.3 ODPG: relationships and graph structure

The Open Data Product Graphs (ODPG) specification represents relationships among products, use cases, business objectives, KPIs, systems and other entities. ^{S20} This aligns directly with the argument in this paper that product portfolios are better understood as graphs than as flat inventories. The meaning of a product is partly contained in its relationships: what it consumes, what it produces, which objectives it supports and which downstream products depend on it.

16.4 ODPV: shared vocabulary

The Open Data Product Vocabulary (ODPV) provides shared terms and semantics across the standards family. ^{S20} A shared vocabulary becomes increasingly important when product metadata is traversed by both humans and AI agents because machine consumers need stable concepts rather than labels whose meaning changes between teams or tools.

16.5 ODPR: workflow contracts

The draft Open Data Product Recipes (ODPR) specification adds workflow contracts for repeatable validation, generation, localization, review, release and other delivery flows. ^{S20 S22} This moves the standards family beyond static description toward repeatable product operations. In practice, it means that product metadata can begin to participate in the lifecycle itself rather than serving only as documentation after the fact.

16.6 AI-agent-first orientation

The standards family explicitly describes itself as AI Agent First and is designed so that humans, platforms and AI agents can understand how data products are defined, organized, connected and interpreted. ^{S20} This orientation is relevant to the distinction developed in this paper because agents need reliable information context before they can act. Productized information gives an agent structured knowledge about what data means, how it is accessed, what constraints apply, who owns it and how it relates to business objectives.

The standards family therefore operates primarily on the data-product side of the distinction while providing machine-readable context that AI products and agents can consume. It does not eliminate the need for an Inference Contract because machine-readable product metadata cannot prove that an AI system behaves correctly. Instead, it strengthens the information side of the relationship so that AI systems have a clearer, more governable context in which to operate.

AI Agents Make the Distinction More Important

Agentic systems make the separation between information reliability and behavioural reliability more important because an agent both depends on context and produces behaviour. The quality of what the agent knows and the quality of what the agent does are related, but they are not the same problem. Treating them as one undifferentiated “AI quality” issue makes diagnosis and governance harder.

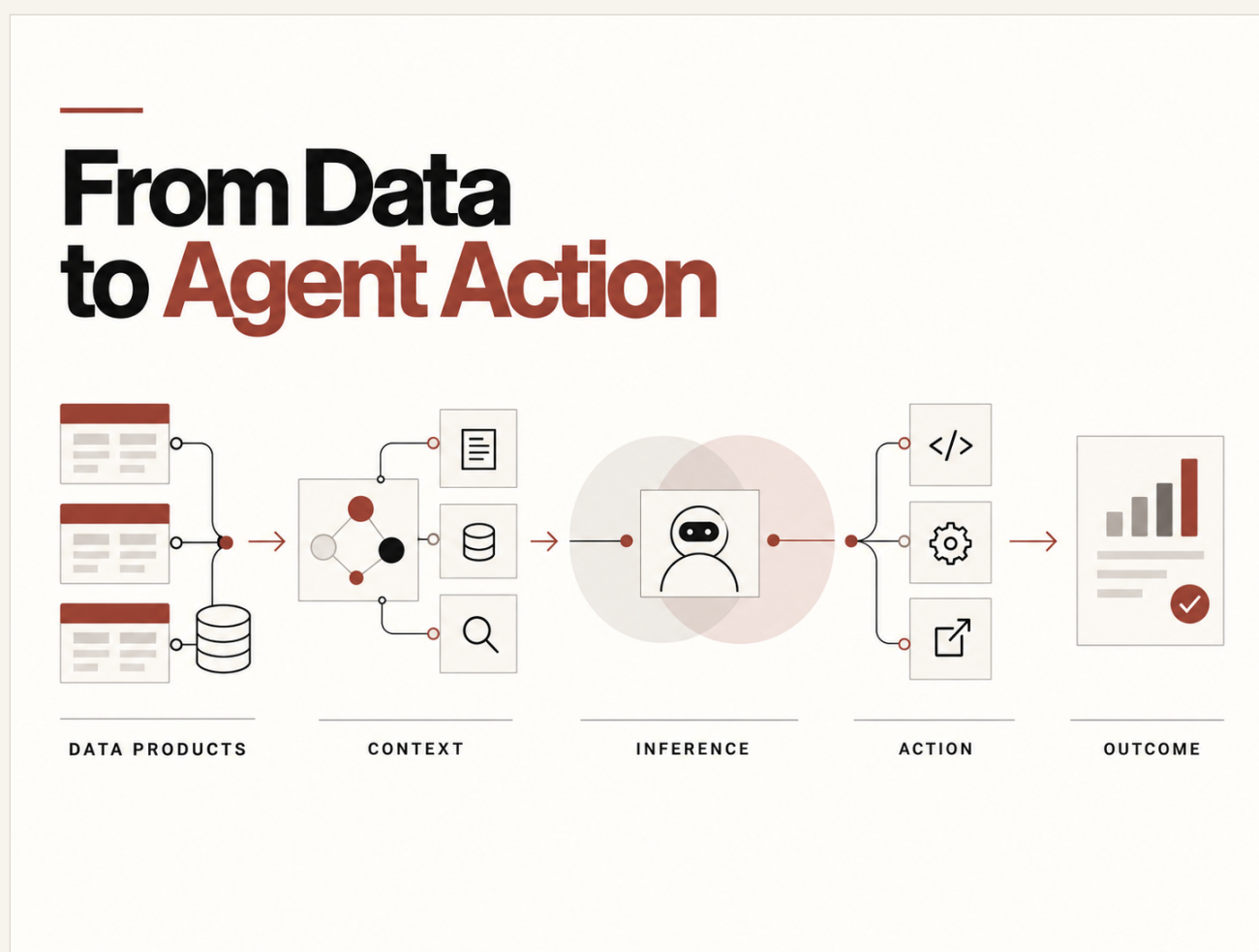


Figure 3. Agentic value chain: data products supply context, inference produces behaviour, and actions create outcomes.

17.1 What the agent knows

An AI agent may depend on customer information, product catalogs, policies, contracts, knowledge documents, business objectives, KPIs, operational state, permissions, tool descriptions and historical interactions. The quality of this context affects every later step, but the agent often encounters it indirectly through retrieval, APIs, semantic layers or product interfaces. If the underlying information is stale, ambiguous, inaccessible or incorrectly governed, even a strong reasoning model starts from a weak foundation.

This dependency gives data products a distinct role in agent architectures. Rather than exposing a collection of undocumented assets, productized information can provide meaning, ownership, quality expectations, access rules and relationships that an agent or orchestration layer can interpret. In an agent-first environment, discoverability and machine-readable semantics become operating requirements rather than catalog conveniences.

17.2 What the agent does

The agent then interprets the request, retrieves information, reasons about options, selects tools, generates outputs, invokes actions, observes results and decides whether to continue. Each stage introduces a different failure possibility. Correct source data may still be retrieved poorly; correct retrieval may still lead to weak reasoning; good reasoning may still result in the wrong tool selection; correct tool selection may fail because authorization is missing; and a technically successful action may still produce a poor business outcome.

This gives the architecture a layered reliability model: information contract , context assembly , inference contract , action contract , business outcome. Productized and machine-readable data context strengthens the first part of the chain, while AI-product evaluation, safety and governance strengthen the inference and action stages. The business outcome provides the final test of whether the complete system creates value.

17.3 Why agent-ready data products matter

An agent needs more than a table name. It needs to know what a product represents, why it exists, who owns it, how it can be accessed, whether it is trustworthy, what its quality measures mean, which constraints apply and how the product relates to the current task. It may also need to understand licensing or permitted-use conditions and compare one potential data product with another before choosing an information source.

This requirement helps explain why the Open Data Products standards family has expanded beyond a single product specification toward catalogs, graph relationships, shared vocabulary and workflow contracts. ^{S20} The expansion reflects a wider move from human-readable documentation toward machine-actionable product context. In agent architectures, that shift is not merely about improving search; it is about making information dependencies explicit enough that automated systems can reason about them under governance.

From Separate Inventories to Connected Product Portfolios

Many organizations maintain separate inventories for different technical and management concerns. A data catalog lists data assets, an AI registry lists models, a project system lists use cases, strategy documents list objectives, governance systems list controls, application portfolios list systems and product tools list roadmaps. Each inventory may be useful on its own, but together they fragment the value chain and make it difficult to understand why products exist and how they depend on one another.

The fragmentation becomes visible when leaders ask straightforward questions. They may need to know which data products support a particular AI product, which AI products depend on a customer domain, which business objective justifies continued investment in a data product, which use cases would fail if a product were retired, or which agents consume information that lacks a clear governance contract. They may also need to know which AI products generate outputs that downstream systems have effectively turned into reusable data products. Separate inventories answer these questions only through manual reconciliation.

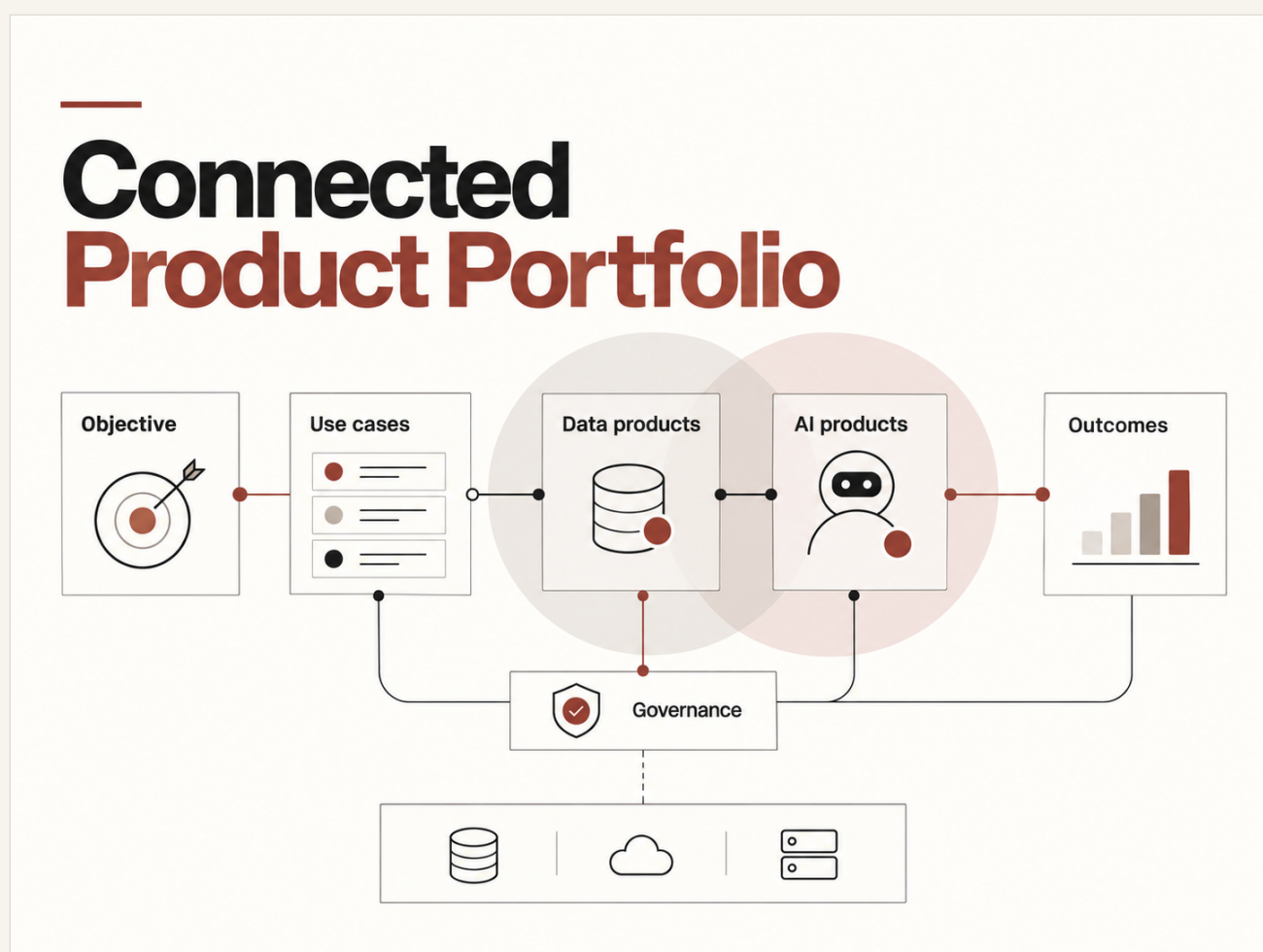


Figure 4. Connected Product Portfolio: objectives, use cases, data products, AI products, governance and outcomes belong in one relationship model.

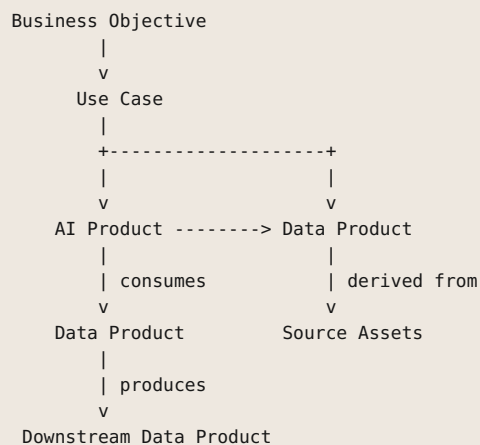
18.1 Proposed portfolio entities

A connected product portfolio therefore needs to represent business objectives, outcomes, KPIs, use cases, AI products, data products, conventional digital products, agents, systems, data assets, models, governance profiles, owners and signals within one relationship model. Not every organization needs every entity type, but the portfolio should preserve the distinctions that change decisions. The purpose is not to create a universal enterprise metamodel; it is to make value, dependency and accountability traceable.

18.2 Proposed relationship types

Relationships such as *supports*, *consumes*, *produces*, *depends on*, *enables*, *governed by*, *owned by*, *measured by*, *exposed through*, *derived from*, *used by* and *replaced by* often carry more management value than the classification label alone. A product becomes strategically understandable when leaders can see the objectives it supports, the products it depends on, the consumers that rely on it and the controls that govern its use.

18.3 A basic portfolio graph



The graph prevents two opposite errors. One is treating every dependency as part of a single giant product, which hides distinct contracts and accountability. The other is treating every technical artifact as an independent product, which inflates portfolios and weakens the meaning of ownership. A connected model preserves product boundaries while making dependencies visible.

Maysano as an Implementation Example of the Connected Model

A white paper that defines AI products and data products should not use the author's own platform as evidence for the definitions. Maysano is therefore included only as an implementation example showing how the connected portfolio model described above can be operationalized. Its public positioning describes an operating layer that connects business outcomes, data products, governance and AI while maintaining traceability from objectives and use cases into structured product portfolios and execution. [S23](#) [S24](#)

Maysano Portfolio Studio uses the Open Data Products standards family to turn existing business material, including presentations, documents, spreadsheets, objectives and use cases, into structured product portfolios for review and delivery. Its public product description emphasizes portability because the meaning of the generated portfolio is represented through open specifications rather than being trapped solely inside the application. [S24](#) This is relevant to the argument in this paper because it demonstrates how product contracts and relationships can be made machine-readable without using AI itself as the data model.

The implementation separates concerns in a way that is consistent with the Two Product Contracts model. Business objectives, use cases, products and delivery context remain connected rather than being reduced to a list of datasets. The Open Data Products standards family provides the machine-readable representation layer, while AI-assisted workflows support portfolio analysis, product generation, reasoning and operational tasks on top of that structure. The example therefore shows cooperation between data-product and AI capabilities without requiring the categories to collapse into one another.

When the Distinction Matters

Taxonomy earns its place only when it changes a decision. The distinction between AI products and data products is therefore most useful when it affects discovery, architecture, ownership, metrics, governance or investment. If the label does not change any of these, the organization gains little from maintaining a separate category.

20.1 Product discovery

Data-product discovery begins with recurring information needs. Teams need to understand who consumes the information, which semantics matter, what form of access works, which level of quality is necessary and where reuse across consumers is realistic. AI-product discovery includes those information questions but adds another layer: whether inference is appropriate for the problem, which task the AI should perform, how much error the workflow can tolerate, how uncertainty should be communicated, which decisions require human oversight and how behaviour will be evaluated in context. The distinction therefore changes the questions a product team asks before it builds anything.

20.2 Architecture

The architectural emphasis also differs. Data-product architecture is dominated by schemas, transformations, semantics, output ports, APIs, events, storage, lineage, quality controls and access. AI-product architecture adds models, prompts, feature systems, retrieval, vector infrastructure, orchestration, tool use, inference endpoints, evaluations and safety controls. Many real systems need both sets of capabilities, but making the product contracts explicit helps teams decide which interfaces deserve independent service levels and which components remain implementation details.

20.3 Ownership, metrics and governance

Ownership can follow the same pattern. A domain team may own the information product while an AI-product team owns the inference experience, or one cross-functional product team may own both contracts. The important issue is not the number of teams but whether the responsibilities are explicit. Metrics follow the same logic: data-product scorecards should not be identical to AI-product scorecards because information reliability and behavioural reliability are different. Governance likewise overlaps without becoming identical, because the organization must control both the use of information and the consequences of inferred behaviour.

20.4 Portfolio investment

The distinction also informs investment decisions. Executives need to know whether a proposed investment creates reusable information infrastructure, inference capability, or both. A reusable customer data product may justify funding because many downstream products depend on it, while an AI agent may justify funding through task automation or improved decisions. Understanding which value mechanism is being funded improves prioritization, cost allocation and dependency risk management.

When the Distinction Adds Little Value

There are situations in which forcing a separate AI-product or data-product identity creates more overhead than clarity. The distinction adds little when there is no independent consumer relationship, no separate service commitment, no separate roadmap, no distinct risk treatment, no useful metric set and no ownership decision that would change as a result of the label. In those cases, a component or asset description is often more accurate and easier to operate.

A recommendation algorithm may remain an AI component inside a commerce product when consumers never interact with it independently and the team does not maintain a separate inference-service contract. A small lookup table may remain an internal data asset when no other team intentionally consumes it. An embedding index can remain part of an AI-product architecture, and a model-output table used only inside one service can remain an implementation component. None of these decisions diminishes the technical importance of the artifact; they simply avoid turning every internal dependency into a product.

The practical rule is that product taxonomy should support management rather than the reverse. If a classification does not change ownership, service expectations, governance, investment, roadmap or measurement, the organization should be cautious about adding another product label. The goal is precision where precision improves decisions, not maximal categorization.

Decision Framework

The decision framework developed in this paper starts with product qualification rather than with AI or data. An initiative should first be tested for identifiable consumers, a recurring need, accountable ownership, supported interfaces, measurable quality or service expectations and an ongoing lifecycle. If those conditions are largely absent, the initiative is more accurately described as an asset, component, feature, experiment or project, and further product classification is premature.

The second step is to test for a Data Consumption Contract. The key question is whether consumers intentionally use the information as an offering with independent value. Maintained semantics and interfaces, explicit quality and freshness expectations, defined access and a product that remains useful when AI is removed all strengthen the case for classification as a data product. The emphasis is not on the existence of data, because every software product has data, but on the deliberate productization of information consumption.

The third step is to test for an Inference Contract. If core value depends on prediction, recommendation, classification, generation, reasoning or AI-selected action, and if the product promise materially collapses when inference is removed, there is strong evidence for an AI product. The case becomes stronger when behaviour requires explicit evaluation, monitoring and lifecycle controls because model-related changes have operational consequences for consumers.

When both contracts are independently meaningful, the correct outcome is both rather than an attempt to choose a winner. Dual classification does not dictate dual teams or duplicate governance structures; it merely records that the initiative carries two product promises. The final management decision concerns boundaries: whether one team should own both contracts, whether separate SLOs or roadmaps are useful, whether governance gates or catalog entries should differ, and whether the consumer groups are the same. Classification and organizational design are related decisions, but they are not the same decision.

Worked Examples

23.1 Customer 360 and next-best action

Consider a customer-retention initiative that combines a Customer 360 product with a next-best-action service. The Customer 360 layer exposes governed customer identity, interactions, products, consent and relevant history to service, marketing, analytics and other systems. Its value survives independently of the recommendation engine because those consumers still need a trusted customer representation. The next-best-action service adds a different promise by inferring which action is most suitable for a particular interaction. The initiative is therefore best classified as both when the recommendation capability is independently managed.

The distinction also improves measurement. The data product can be assessed through profile completeness, identity-match quality, freshness and API availability, while the AI product can be assessed through recommendation acceptance, uplift, model quality, latency and the rate of harmful or inappropriate recommendations. Both sets of measures should ultimately connect to the shared outcome of improved retention or service relevance, but merging them into one score would make diagnosis harder.

23.2 Fraud intelligence

A fraud-intelligence platform provides another example of dual contracts. Reusable transaction, account, merchant, entity and label information can form governed data products that support several analytical and operational consumers. Fraud risk scoring and decision support then add inference-based behaviour on top of those inputs. The classification is both because the information products remain useful beyond the scoring service and the scoring service has its own quality and operational commitments.

The failure modes demonstrate why the distinction matters. The model can be statistically sound while an upstream merchant classification is wrong, producing a poor decision from incorrect context. Conversely, the underlying data can be correct while a threshold or calibration choice creates unacceptable false positives. Separate product contracts make it easier to identify whether remediation belongs in the information layer, the inference layer or the surrounding business rule.

23.3 Demand forecasting

Demand forecasting illustrates a complete product chain. Historical sales, promotions, inventory, product hierarchy and calendar information may enter as data products. A forecasting service then generates future demand estimates and qualifies as an AI product because inference is central to the value it provides. Those forecasts may then be published by SKU, location and horizon through a governed interface for planning, procurement and logistics. Once downstream consumers rely on that publication as maintained information, the output becomes another data-product boundary.

The architecture is therefore data products, AI product, data product. This pattern is important because it shows why separate AI and data product portfolios often provide an incomplete picture. The value chain crosses the boundary repeatedly, and the portfolio needs to represent those relationships rather than force the whole initiative into one category.

23.4 Enterprise knowledge assistant

An enterprise knowledge assistant may also qualify as both, but the answer depends on the product boundary. If the organization maintains a curated, permission-aware knowledge corpus with document metadata and structured policy or product information for several consumers, that layer has a credible Data Consumption Contract. The assistant then adds question answering, synthesis, retrieval orchestration and tool use, creating an Inference Contract on top of the information layer.

If the knowledge index exists only as an internal implementation detail for the assistant and has no independent consumers, roadmap or service commitments, the primary classification may remain AI product. This example shows why the presence of a retrieval index or vector database is insufficient evidence of a data product. Product status comes from the consumer relationship, not the storage technology.

23.5 Coding copilot

A coding copilot is typically an AI product because code generation, explanation, completion and transformation constitute the core value proposition. The architecture may contain repository context, prompt history, telemetry, embeddings and other forms of data, but those assets do not become products merely because the system depends on them. Unless another consumer intentionally uses them through a managed offering, they remain data assets or components inside the AI-product architecture.

23.6 Open-data portal with productized use cases

An open-data portal provides a useful example from the public-data domain. A conventional portal exposes datasets and metadata, but a data-product approach groups and describes those datasets around recurring public, business or policy use cases, adds clearer value propositions, quality expectations, access information and related assets, and treats the resulting offering as something consumers can intentionally discover and reuse. In this configuration, the published information remains a data product even if AI assists behind the scenes with generating candidate use cases, producing structured product descriptions or matching related datasets.

The classification changes only when inference becomes a separate consumer promise. If the portal also offers an AI assistant whose primary value is reasoning over public data, answering questions or completing tasks, that assistant may be a separate AI product consuming the data products. The example again shows why implementation technology should not determine the label: AI can assist the creation or discovery of a data product without turning the data product itself into an AI product.

Organizational Language That Reduces Confusion

Organizations benefit from a compact vocabulary that separates systems, components, products and assets without forcing every object into a binary AI-versus-data classification. An AI system is the machine-based technical system that infers outputs from inputs in line with OECD, NIST and related definitions. An AI component is an AI capability embedded inside another product without its own product contract, while an AI-enabled product is a broader product whose value is enhanced by AI but whose primary identity can survive without it. An AI product is reserved for cases in which inference-based behaviour materially defines the primary product promise.

The data side benefits from the same discipline. A data asset is a dataset, table, event stream, metric, file or other information resource, while a data-product component is an asset or capability that forms part of a data product without carrying an independent consumer contract. A data product is the managed offering for recurring data or information consumption. The classification both applies when a Data Consumption Contract and an Inference Contract are independently meaningful within the same initiative, while neither yet identifies artifacts that have not established a durable product relationship.

This vocabulary is intentionally modest. It does not attempt to replace existing enterprise architecture or data-governance taxonomies. Its purpose is to prevent category errors that create operational confusion, such as calling every model an AI product or every table a data product. Clear language should make accountability easier, not create another taxonomy that teams need to manage for its own sake.

Implications for Product Portfolios

The final implication of this analysis is organizational. A modern enterprise portfolio should not be divided into disconnected “AI portfolio” and “data product portfolio” universes because the products depend on each other too strongly. A useful portfolio needs to show value, dependency and accountability together: which objective and use case justify each product, which AI products, data products, systems and assets it consumes or produces, and who owns the commitments by which success is measured.

A connected model also makes reuse visible. A high-value data product may support ten AI products, while a strategically important AI product may depend on five domain data products. A model registry alone cannot show that business relationship, a data catalog alone cannot show the behavioural layer, and a project portfolio alone cannot show the technical dependency chain. A graph-oriented product portfolio can preserve these distinctions while connecting them into one view.

The Open Data Products standards family moves in this direction by separating individual product description, catalog and portfolio structure, graph relationships, shared vocabulary and workflow contracts. ^{S20} Maysano provides one implementation example of connecting that structure to business objectives and AI-assisted portfolio workflows. ^{S23 S24} The broader principle is independent of any tool: product portfolios become more useful when they represent not only what exists, but also how products contribute to outcomes and depend on one another.

Final Position

The distinction between AI products and data products is best understood as contractual and operational rather than purely technological. A data product commits to making information dependable for recurring consumption. An AI product commits to making inference dependable enough to create a useful outcome. The two share product-management principles, infrastructure, governance concerns and data dependencies, and they often sit inside the same initiative. They should therefore not be managed as unrelated disciplines, but they should not be collapsed into one concept either.

The distinction remains useful because the failure modes, lifecycle controls, quality evidence, governance questions, economics and consumer promises differ. The strongest practical shorthand is therefore that a data product productizes data consumption, an AI product productizes inference, and an initiative classified as both independently productizes information consumption and inference. These formulations do not attempt to settle every possible industry definition. They provide an operating boundary that tells teams what they need to own, measure and govern.

For decision makers, the practical questions are more important than the label itself. They need to understand what the consumer depends on, what value promise the organization is operating, what fails when that promise is broken, who owns it, how success is measured, which governance controls apply and which other products are consumed or produced along the way. Once those relationships are explicit, the apparent contradiction between AI products and data products becomes manageable.

The organization therefore does not need one universal taxonomy that forces every initiative into a single box. It needs a connected product model that shows where trusted information has been productized, where inference has been productized, where both contracts coexist and how those products combine to create measurable outcomes. That model provides a stronger foundation for product management, architecture, governance and portfolio investment than technology labels alone.

Appendix A. Detailed Comparison Matrix

Dimension	Data product	AI product	Both
Core value	Reliable information consumption	Useful inferred behaviour	Information and inferred behaviour
Primary promise	Give me trusted reusable information	Predict, generate, recommend, classify, reason or act	Provide both
Primary interface	Table, API, file, event, graph, semantic model	App, chat, scoring API, generative API, agent interface	Multiple interfaces
Consumer	Analyst, application, team, model, agent	End user, application, workflow, agent	Mixed
Main quality object	Data and semantics	Behaviour and outcome	Both
Typical uncertainty	Freshness, completeness, correctness, meaning	Model error, hallucination, drift, calibration, safety	Combined
Lifecycle trigger	Source/schema/semantic/contract change	Model/prompt/retrieval/eval change	Either
Dominant governance	Access, lineage, quality, retention, interoperability	AI risk, evals, human oversight, safety, model provenance	Integrated
Main SLOs	Availability, freshness, quality, contract adherence	Availability, latency, eval quality, safety, cost	Separate plus shared outcome
Reuse pattern	Many consumers reuse information	Many users/workflows reuse behaviour	Shared value chain
Failure question	Was the information wrong or unusable?	Did the AI behave incorrectly?	Which layer failed?
Removal test	Survives removal of AI	Loses central value without inference	Both tests positive

Appendix B. Product Qualification Checklist

Use this checklist before applying either product label.

- | | |
|--|--|
| ☐ Named consumer groups exist. | ☐ Lifecycle states are managed. |
| ☐ A recurring need exists. | ☐ Consumers receive change communication. |
| ☐ A product owner or accountable team exists. | ☐ Usage is monitored. |
| ☐ A value proposition is documented. | ☐ Outcomes are measured. |
| ☐ Supported interfaces exist. | ☐ Feedback influences the roadmap. |
| ☐ Quality expectations are measurable. | ☐ Governance requirements are known. |
| ☐ Service expectations are defined. | ☐ Retirement criteria exist. |

If most boxes remain empty, the object is more accurately described as an asset, feature, model, experiment or project.

Appendix C. Data Consumption Contract Checklist

- | | |
|---|--|
| ☐ Consumer groups | ☐ Provenance |
| ☐ Recurring information need | ☐ Ownership |
| ☐ Value proposition | ☐ Access controls |
| ☐ Product scope | ☐ License or usage rights |
| ☐ Semantics | ☐ Privacy requirements |
| ☐ Schema | ☐ Versioning |
| ☐ Interfaces | ☐ Breaking-change policy |
| ☐ Data quality expectations | ☐ Support |
| ☐ Freshness | ☐ Usage measures |
| ☐ Availability | ☐ Outcome measures |
| ☐ Lineage | |
-

Appendix D. Inference Contract Checklist

- | | |
|---|--|
| ☐ Intended task | ☐ Fallback behaviour |
| ☐ Intended users | ☐ Latency target |
| ☐ Intended context | ☐ Availability |
| ☐ Supported inputs | ☐ Cost target |
| ☐ Expected outputs | ☐ Model provenance |
| ☐ Evaluation dataset | ☐ Prompt/configuration versioning |
| ☐ Evaluation criteria | ☐ Retrieval versioning where relevant |
| ☐ Acceptable error | ☐ Monitoring |
| ☐ Safety constraints | ☐ Incident process |
| ☐ Human oversight | ☐ Change approval |
| ☐ Tool permissions | ☐ Outcome measures |
| ☐ Grounding requirements | |
-

Appendix E. Source Notes and Evidence Base

The following references form the main evidence base for this white paper. They are grouped by role rather than alphabetically so readers can see which claims they support.

Formal and authoritative AI definitions

S01 OECD, updated definition of an AI system

Organisation for Economic Co-operation and Development. *Updates to the OECD's definition of an AI system explained*. Updated definition adopted in 2023.

<https://oecd.ai/en/wonk/ai-system-definition-update>

Why it matters: provides an intergovernmental definition centred on inference, inputs, outputs, autonomy and adaptiveness.

S02 NIST, AI Risk Management Framework 1.0

Tabassi, E. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1.

<https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10>

Why it matters: defines the AI-system context and provides lifecycle trustworthiness and risk-management concepts.

S03 European Union, AI Act

Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence.

<https://eur-lex.europa.eu/eli/reg/2024/1689/oj>

Why it matters: provides a legal AI-system definition and formal treatment of AI systems in relation to products, deployers, intended purpose and safety components.

S04 NIST, AI RMF Resource Center and Framework guidance

<https://airc.nist.gov/airmf-resources/airmf/>

Why it matters: explicitly applies trustworthiness and lifecycle thinking to AI products, services and systems.

S05 ISO/IEC 42001:2023

International Organization for Standardization. *Information technology — Artificial intelligence — Management system*.

<https://www.iso.org/standard/42001.html>

Why it matters: international management-system standard for organizations developing, providing or using AI-based products and services.

S06 ISO/IEC 22989:2022

International Organization for Standardization. *Artificial intelligence — Artificial intelligence concepts and terminology*.

<https://www.iso.org/standard/74296.html>

Why it matters: terminology baseline for AI concepts.

Peer-reviewed and academic data-product evidence

S07 Blohm, Wortmann, Legner et al. (2024)

Blohm, I., Wortmann, F., Legner, C., et al. *Data products, data mesh, and data fabric: New paradigm(s) for data and analytics?* Business & Information Systems Engineering, 66, 643–652.

<https://link.springer.com/article/10.1007/s12599-024-00876-5>

Why it matters: peer-reviewed synthesis of data products, data mesh and data fabric. It emphasizes data consumption and recognizes data products ranging from datasets to predictive and prescriptive analytics.

S08 Hasan, Finkel and Legner (2026)

Hasan, M. R., Finkel, B., & Legner, C. *The Data Product Canvas: Designing Data Products for Sustained Value From Enterprise Data*. Information Systems Journal, 36(1), 144–158.

<https://onlinelibrary.wiley.com/doi/10.1111/isj.12603>

Why it matters: empirical and design-science work connecting data-product design with consumers, value, desirability, feasibility and viability.

S19 Wang (1998)

Wang, R. Y. *A Product Perspective on Total Data Quality Management*. Communications of the ACM.

Why it matters: early information-product thinking that predates data mesh and frames information quality around consumers and product thinking.

Data mesh and major industry definitions

S09 Dehghani (2020)

Dehghani, Z. *Data Mesh Principles and Logical Architecture*.

<https://martinfowler.com/articles/data-mesh-principles.html>

Why it matters: foundational data-mesh source defining data as a product as one of four core principles.

S10 AWS, What is a Data Mesh?

<https://aws.amazon.com/what-is/data-mesh/>

Why it matters: gives a narrow operational definition of a data product as a structured, reusable, independently usable component serving a business purpose.

S11 IBM, What is a Data Product?

<https://www.ibm.com/think/topics/data-product>

Why it matters: provides a broader data-product definition and explicitly permits datasets, dashboards, reports, ML models, queries and pipelines as components.

S12 DataHub, Data Products

DataHub product and metamodel guidance.

<https://docs.datahub.com/docs/features/feature-guides/data-products/>

Why it matters: operationalizes data products as curated, discoverable, governed collections of data assets.

Production AI and lifecycle evidence

S14 Google Cloud, MLOps

MLOps: Continuous delivery and automation pipelines in machine learning.

<https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>

Why it matters: demonstrates that production ML requires far more than model code and depends on validation, serving, metadata, automation and monitoring.

S15 Microsoft Azure Architecture Center, MLOps v2

<https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/machine-learning-operations-v2>

Why it matters: documents production monitoring concerns including model degradation, data drift, prediction drift, governance, security and cost.

S16 Google PAIR, People + AI Guidebook

<https://pair.withgoogle.com/guidebook/>

Why it matters: human-centred AI product guidance covering user needs, mental models, trust, feedback and AI failure.

S25 Sculley et al. (2015)

Sculley, D., Holt, G., Golovin, D., et al. *Hidden Technical Debt in Machine Learning Systems*. NIPS 2015.

<https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>

Why it matters: foundational evidence that production ML systems accumulate distinct forms of technical debt through data dependencies, feedback loops and changing environments.

Product-management baseline

S17 Cagan, Silicon Valley Product Group

Cagan, M. *What is a Product?*

<https://www.svpvg.com/what-is-a-product/>

Why it matters: customer, business and technology framing for product thinking.

S18 Torres, Product Talk

Torres, T. Product discovery and outcome-oriented product-management materials.

<https://www.producttalk.org/>

Why it matters: continuous discovery and outcome orientation provide the baseline separating durable products from one-time technical deliveries.

Open Data Products standards family

S20 Open Data Products standards family

Linux Foundation project site.

<https://opendataproductions.org/>

Why it matters: current public definition of the AI Agent First Open Data Products standards family, including ODPS, ODPC, ODPG, ODPV, draft ODPR and the SDK. The site also documents current maintainer roles and the author's role as igniter.

S21 Open Data Product Specification 4.1

<https://opendataproductions.org/v4.1/>

Why it matters: current production ODPS specification and machine-readable data-product model.

S22 Open Data Product Recipes 1.0 Draft

<https://opendataproductions.org/odpr-v1.0/>

Why it matters: workflow-contract layer for repeatable human, platform and AI-agent delivery flows.

Implementation example

S23 Maysano

<https://maysano.com/>

Why it matters: public description of Maysano as an operating layer connecting business objectives, data products, governance and AI.

S24 Maysano Portfolio Studio

<https://studio.maysano.com/>

Why it matters: implementation example showing conversion of business material into structured data-product portfolios using the Open Data Products standards family.

Dr. Jarkko Moilanen

AI Product Leadership from Strategy to Working Systems

Dr. Jarkko Moilanen works across AI products, data products, open standards and product portfolio management. His work connects business objectives with working AI systems, agentic architectures, machine-readable product definitions and the operating models required to manage them.

He ignited the Open Data Product Specification initiative, which later expanded into today's Linux Foundation Open Data Products standards family. The family now covers product definition, catalogs and portfolios, graph relationships, shared vocabulary and workflow contracts for human, platform and AI-agent use.

This white paper separates external evidence, the author's synthesis, and implementation examples. The Two Product Contracts model and the Removal Test are proposed frameworks built from the wider evidence base reviewed in this paper.

FOCUS

- AI products
- Data products
- AI agents
- Open standards
- Product portfolios

MORE

jarkkomoilanen.com

Strategy. Product. Architecture. Working systems.